


**Pontificia Universidad Javeriana**
Facultad de Ingeniería, Ingeniería de Sistemas

## Software Requirements Specification
**SRS**

# GammaLab v2.0

| | |
|---|---|
| **Proyecto** | Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab |
| **Documento** | Software Requirements Specification (SRS) |
| **Versión** | 1.0 |
| **Estado** | En elaboración |
| **Fecha** | 27 de agosto de 2026 |

**Autores**

Juan Felipe Romero Ortiz (juanromeroo@javeriana.edu.co)
Laura Juliana Garzón Arias (garzonlj@javeriana.edu.co)
Xiomara Herrera Suárez (x_herrera@javeriana.edu.co)
Sebastián Almanza Galvis (s.almanzag@javeriana.edu.co)

**Nota de uso.** Documento académico/interno del equipo de proyecto. No distribuir sin autorización.

# HISTORIAL DE CAMBIOS

| **Versión** | **Fecha** | **Sección** | **Descripción** | **Responsable** |
|---|---|---|---|---|
| v1.0 | 17/05/26 inicio del documento | Se inició la creación del SRS y se dividió el trabajo para cada persona | Juan Felipe Romero Ortiz, Laura Juliana Garzón Arias, Xiomara Herrera Suárez, Sebastián Almanza Galvis | |

Tabla 1: Historial de cambios

# CONTENIDO

**HISTORIAL DE CAMBIOS** **1**

**CONTENIDO** **2**

**LISTA DE TABLAS** **4**

**LISTA DE ILUSTRACIONES** **5**

**1 INTRODUCCIÓN** **6**
1.1 Propósito . . . 6
1.1.1 Contexto - GammaLab v1.0 . . . 6
1.2 Alcance . . . 7
1.3 Definiciones, Acrónimos y Abreviaciones . . . 8
1.4 Referencias . . . 9
1.5 Apreciación Global . . . 10

**2 DESCRIPCIÓN GLOBAL** **12**
2.1 Perspectiva del Producto . . . 12
2.1.1 Interfaces con el Sistema . . . 12
2.1.2 Interfaces con el Usuario . . . 12
2.1.3 Interfaces con el Hardware . . . 13
2.1.4 Interfaces con el Software . . . 13
2.1.5 Interfaces de Comunicación . . . 14
2.1.6 Restricciones de Memoria . . . 14
2.1.7 Operaciones . . . 16
2.1.8 Periodos de Actividad e Inactividad . . . 16
2.1.9 Tratamiento de Sesiones y Persistencia . . . 17
2.1.10 Funciones de Soporte al Procesamiento . . . 17
2.1.11 Requerimientos de Adaptación del Sitio . . . 17
2.2 Funciones del Producto . . . 18
2.2.1 Pantalla general . . . 18
2.2.2 Home . . . 19
2.2.3 Preprocessing . . . 19
2.2.4 Analysis . . . 19
2.3 Características del Usuario . . . 20
2.4 Restricciones . . . 21
2.4.1 Restricciones Generales . . . 21
2.4.2 Restricciones de Software . . . 22
2.4.3 Restricciones de Hardware . . . 22
2.5 Modelo del dominio . . . 22
2.5.1 DataStore . . . 25
2.5.2 SignalDataset . . . 25
2.5.3 TrialDataset . . . 26
2.5.4 CouplingResult . . . 26

2.5.5 ProjectSession . . . 27
2.6 Suposiciones y dependencias . . . 27
2.6.1 Suposiciones técnicas . . . 27
2.6.2 Suposiciones sobre el usuario . . . 28
2.6.3 Dependencias externas y del sistema . . . 28

**3 REQUERIMIENTOS ESPECÍFICOS 29**
3.1 Nomenclatura de Requerimientos . . . 29
3.2 Requerimientos Funcionales . . . 30
3.2.1 Módulo IO: Entrada/Salida . . . 30
3.2.2 Módulo UI: Interacción general . . . 31
3.2.3 Módulo WV: Wavelet/Escalograma . . . 32
3.2.4 Módulo PAC: Acoplamiento fase-amplitud . . . 32
3.3 Requerimientos No Funcionales . . . 34
3.3.1 Categoría U: Usabilidad . . . 34
3.3.2 Categoría P: Rendimiento . . . 35
3.3.3 Categoría R: Confiabilidad . . . 37
3.3.4 Categoría S: Soporte . . . 37

**4 PROCESO INGENIERÍA DE REQUERIMIENTOS 38**
4.1 Enfoque General . . . 38
4.2 Actividades del Proceso . . . 39
4.3 Control de Cambios . . . 39
4.4 Artefactos Generados . . . 40

**5 PROCESO VERIFICACIÓN 41**
5.1 Participación del Cliente . . . 41
5.2 Dinámica de Verificación . . . 41
5.3 Mecanismo de Control y Seguimiento . . . 41
5.4 Criterios de Verificación por Tipo de Requerimiento . . . 42
5.5 Resultado del Proceso . . . 44

**6 ANEXOS 45**

# Índice de tablas


1 Historial de cambios . . . . . 1
2 Acrónimos y abreviaciones en el SRS . . . . . 9
3 Características generales de los usuarios de GammaLab 2.0 . . . . . 21
4 Entidad de dominio: DataStore . . . . . 25
5 Entidad de dominio: SignalDataset . . . . . 25
6 Entidad de dominio: TrialDataset . . . . . 26
7 Entidad de dominio: CouplingResult . . . . . 26
8 Entidad de dominio: ProjectSession . . . . . 27
10 Requerimientos funcionales del módulo IO . . . . . 31
12 Requerimientos funcionales del módulo UI . . . . . 32
14 Requerimientos funcionales del módulo WV . . . . . 32
16 Requerimientos funcionales del módulo PAC . . . . . 34
18 Requerimientos no funcionales de la categoría Usabilidad . . . . . 35
20 Requerimientos no funcionales de la categoría Rendimiento . . . . . 37
22 Requerimientos no funcionales de la categoría Confiabilidad . . . . . 37
24 Requerimientos no funcionales de la categoría Soporte . . . . . 38
25 Artefactos generados durante el proceso de ingeniería de requerimientos . . . 40
26 Criterios de verificación por tipo de requerimiento . . . . . 43

# Índice de figuras

1 Modelo conceptual del dominio de Gamma Lab 2.0. Las entidades representan los componentes centrales de almacenamiento, procesamiento y persistencia de datos neurofisiológicos. . . . . 24

# 1. INTRODUCCIÓN

## 1.1. Propósito

El presente documento constituye la Especificación de Requisitos de Software (SRS) de GammaLab 2.0. Con el proposito de brindar una orientación metodológica clara del inicio del ciclo de desarrollo. GammaLab v2.0 es una interfaz gráfica de escritorio de tipo standalone orientada al análisis, procesamiento y visualización de señales electroencefalográficas o EEG. Concebido como una alternativa de código abierto y enfocado en el uso local para el Laboratorio de Electrofisiología de la Universidad de Colombia, el sistema está desarrollado con el lenguaje Python, utilizando el framework PyQt5 y librerías científicas como VTK. Siguiendo con la visión de GammaLab v1.0, su diseño arquitectónico evoluciona con el fin de resolver brechas en el paralelismo de cómputo y añadir nuevos métodos de acoplamiento cruzado de frecuencias (Phase-Amplitude-Coupling o PAC).

Este documento especifica de manera formal, atómica y verificable la totalidad de los requerimientos funcionales y no funcionales que rigen el desarrollo de GammaLab v2.0. El alcance de este abarca detalladamente los requerimientos de interfaz de usuario, gestión de memoria DataStore, el procesamiento asíncrono en segundo plano del Kernel y las especificaciones técnicas del nuevo módulo de análisis (PAC). Se excluye del alcance del SRS las rutas de mantenimiento técnico a largo plazo del software, así mismo el código heredado de GammaLab v1.0 que se conservará sin modificaciones funcionales.

La audiencia de este documento comprende cuatro grupos: el equipo de desarrollo, que lo utilizará como referencia técnica durante el diseño y la implementación; la directora del proyecto, en su rol de Product Owner, quien lo empleará para validar que el alcance acordado se mantenga a lo largo del desarrollo; el Laboratorio de Señales de la Universidad Nacional de Colombia, como cliente y usuario final del sistema; y los evaluadores académicos del proyecto de grado, ante quienes el documento evidencia el rigor metodológico del proceso de ingeniería seguido.

La importancia de este SRS radica en que este actúa como puente de información oficial entre las necesidades del laboratorio y la implementación técnica del software. Al definir los criterios de medición para cada funcionalidad y asociarlos a los requerimientos, el documento mitiga el riesgo de ambigüedad técnica. Por último, este texto sirve como un instrumento principal de trazabilidad y verificación que, a lo largo del ciclo de vida del proyecto, garantice que cada incremento dentro del software responda de forma adecuada a las necesidades previamente acordadas.

### 1.1.1. Contexto - GammaLab v1.0

GammaLab nació en 2025 [14] como una alternativa de código abierto a las herramientas con licencia comercial y fragmentada en el mercado, enfocada específicamente en las necesidades del Laboratorio de Electrofisiología de la Universidad Nacional de Colombia. El sistema se estructuró bajo una arquitectura microkernel con un ecosistema de plugins usando Python,

permitiendo el análisis de señales EEG en los dominios de tiempo, frecuencia y tiempo-frecuencia mediante una interfaz gráfica unificada.

Aunque la primera iteración fue validada exitosamente con un 87.5 % de coincidencia frente al sistema de referencia BOARD-FTD-PACC [15], las pruebas de aceptación con el usuario final identificaron brechas críticas para su correcta adopción. Entre estas limitaciones se destacan la ausencia de persistencia de sesiones y gestión de proyectos, la falta de rendimiento al no contar con mecanismos de concurrencia o paralelismo para la distribución de tareas pesadas.

Además, el equipo de investigación reportó la necesidad de contar con un control más preciso sobre la densidad de muestreo y la incorporación de métodos analíticos avanzados. Por todo lo expuesto anteriormente, GammaLab v2.0 se concibe más allá de solo extender estas capacidades, sino para estabilizar la arquitectura meidnate el procesamiento asíncrono en segundo plano, así garantizando la robustez técnica necesaria para los investigadores.

## 1.2. Alcance

El alcance de GammaLab v2.0 lo definen los límites funcionales, técnicos y operativos del sistema, asegurando la entrega de un software robusto y enfocado en el entorno de investigación.

GammaLab v2.0 se define como una aplicación de escritorio standalone, lo que significa que es local y funcional sin acceso a internet. Desarrollada en el lenguaje Python bajo el framework PyQt5 y librerías de visualización científicas VTK. El sistema está diseñado específicamente para interactuar con archivos de señales electroencefalográficas (EEG) en los formatos ABF, EDF y MAT. Su arquitectura está diseñada para investigadores y estudiantes del Laboratorio de Electrofisiología de la Universidad Nacional de Colombia, permitiéndoles ejecutar análisis avanzados sin requerir conocimientos en programación, ni fragmentando su flujo de trabajo en diferentes herramientas durante sus análisis. Con el fin de cerrar las brechas observadas de la primera iteración del software, la visión de esta segunda iteración se definirá por medio de las siguientes funciones:

**Funcionalidades adicionales**

- **Persistencia de sesión y gestión de proyectos:** Implementación del componente ProjectSession para permitir el guardado local, creando una extensión propia para GammaLab (.glab) y la restauración automática del entorno de trabajo.
- **Kernel de procesamiento asíncrono:** Desacoplamiento de la interfaz de usuario mediante un servicio de tareas basado en hilos trabajadores, que ejecuta el análisis matemático pesado en segundo plano sin congelar la GUI.
- **Módulo de análisis PAC:** Incorporación del un plugin especializado para cuantificar el acoplamiento phase-amplitude coupling, permitiendo la selección cruzada de canales fuentes.
- **Optimización visual y gráfica vectorial:** Inclusión de técnicas de submuestreo de señal dinámica para navegación fluida (>= 24 FPS) y soporte de exportación gráfica

en alta calidad mediante el formato EMF.

**Funcionalidades excluidas (fuera de alcance)**

- El sistema no implementará almacenamiento de datos en la nube, sincronización web o bases de datos compartidas. Todo análisis ejecutado en la aplicación se realizará de forma local en el disco duro del usuario.
- El sistema no incluye módulos para la adquisición de señales EEG en tiempo real mediante conexiones físicas a amplificadores o diademas de electrodos.
- Se excluye el soporte para el procesamiento de otro tipo de bioseñales; el soporte es exclusivo de señales electroencefalográficas.

El desarrollo de esta evolución funcional y arquitectónica aporta tres aspectos claves para el ecosistema del laboratorio:

- **Análisis avanzado:** Al generar un entorno puramente visual para métricas complejas de acoplamiento cruzado de secuencias, se elimina la fricción técnica de programar algoritmos desde cero.
- **Soberanía tecnológica y sostenibilidad financiera:** Al ser una alternativa sólida de código abierto bajo la licencia MIT, el laboratorio elimina dependencias de licencias costosas o plataformas comerciales cerradas.
- **Eficiencia computacional:** La paralelización del kernel reduce drásticamente los tiempos de procesamiento de grandes volúmenes de datos, optimizando los tiempos de respuesta del programa.

## 1.3. Definiciones, Acrónimos y Abreviaciones

| Acrónimo / Abreviación | Definición |
|---|---|
| **EEG** | Electroencefalografía (*Electroencephalography*). |
| **GUI** | Interfaz gráfica de usuario (*Graphical User Interface*). |
| **ERP** | Potencial relacionado a eventos (*Event Related Potential*). |
| **FFT** | Transformada Rápida de Fourier (*Fast Fourier Transform*). |
| **PSD** | Densidad Espectral de Potencia (*Power Spectral Density*). |
| **PAC** | Acoplamiento fase-amplitud (*Phase-Amplitude Coupling*). |
| **VTK** | Visualization Toolkit. Librería de visualización científica 2D y 3D. |
| **CSV** | Valores separados por comas (*Comma-Separated Values*). |
| **XLSX** | Formato de hoja de cálculo basado en Office Open XML utilizado por Microsoft Excel. |
| **PNG** | Portable Network Graphics. |
| **JPG** | Joint Photographic Experts Group. |
| **EMF** | Enhanced Metafile Format. |
| **ABF** | Axon Binary File. |
| **EDF** | European Data Format. |
| **MAT** | Formato de archivo de MATLAB. |
| **RAM** | Random Access Memory. |
| **VRAM** | Video Random Access Memory. |
| **CPU** | Central Processing Unit. |
| **I/O** | Input/Output. |
| **UX** | User Experience. |
| **SRS** | Software Requirements Specification. |
| **PMP** | Project Management Plan. |

Tabla 2: Acrónimos y abreviaciones en el SRS

## 1.4. Referencias


[1] Microsoft Corporation, "Windows: product family overview," Available: https://www.microsoft.com/windows. Accessed: 2025.

[2] Python Software Foundation, "The Python tutorial: Python 3.11 documentation," 2023. Available: https://docs.python.org/3/tutorial/index.html. Accessed: 2025.

[3] Riverbank Computing, "PyQt: Python bindings for Qt," 2025. Available: https://www.riverbankcomputing.com/software/pyqt/. Accessed: 2025.

[4] Kitware, "The Visualization Toolkit (VTK)," 2025. Available: https://vtk.org/. Accessed: 2025.

[5] NumPy Developers, "NumPy documentation," 2025. Available: https://numpy.org/doc/stable/. Accessed: 2025.

[6] SciPy Community, "SciPy: fundamental algorithms for scientific computing in Python," 2025. Available: https://scipy.org/. Accessed: 2025.

[7] The pandas Development Team, "pandas: Python data analysis library," 2025. Available: https://pandas.pydata.org/. Accessed: 2025.

[8] G. R. Lee, R. Gommers, F. Wasilewski, K. Wohlfahrt, and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019. Available: https://doi.org/10.21105/joss.01237.

[9] S. W. Harden, "pyabf 2.3.5," 2022. Available: https://pypi.org/project/pyabf/.

[10] Teunis van Beelen, "pyEDFlib," 2025. Available: https://pypi.org/project/pyEDFlib/. Accessed: 2025.

[11] Python Software Foundation, "threading: Thread-based parallelism," Python 3 documentation. Available: https://docs.python.org/3/library/threading.html.

[12] The Qt Company, "QThread Class," Qt 5 documentation. Available: https://doc.qt.io/qt-5/qthread.html.

[13] pytest Development Team, "pytest documentation," 2025. Available: https://docs.pytest.org/en/stable/. Accessed: 2025.

[14] S. Asencio Rodríguez, M. F. González Valencia, J. E. Martínez Clavijo, y G. V. Villalobos Rocha, "Gamma Lab," Memoria de trabajo de grado, Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Pontificia Universidad Javeriana, Bogotá, D.C., Colombia, nov. 2025.

[15] C. Gauthier-Umaña, M. Valderrama, A. Múnera, y M. O. Nava-Mesa, "BOARD-FTD-PACC: a graphical user interface for the synaptic and cross-frequency analysis derived from neural signals," *Brain Informatics*, vol. 10, art. 12, may. 2023, doi: 10.1186/s40708-023-00191-x. Available: https://link.springer.com/article/10.1186/s40708-023-00191-x.

[16] P. Eeles, "Capturing architectural requirements," Rational Software Corporation, nov. 2001. Available: https://www.ibm.com/developerworks/rational/library/4706.html.

## 1.5. Apreciación Global

Este documento especifica los requerimientos de software de GammaLab 2.0, abarcando desde la descripción general del sistema hasta la definición detallada de sus requerimientos funcionales y no funcionales. Se organiza en tres bloques principales: una descripción general que contextualiza el producto, sus usuarios y las condiciones bajo las cuales opera; la especificación de requerimientos funcionales, organizada por módulo; y los requerimientos no funcionales junto con los procesos de ingeniería y verificación aplicados. Cada sección cumple un rol específico: los requerimientos funcionales y no funcionales orientan las decisiones de diseño

e implementación, mientras que el modelo del dominio y la distribución de requerimientos permiten mantener una comprensión uniforme del sistema entre todos los involucrados.

# 2. DESCRIPCIÓN GLOBAL

## 2.1. Perspectiva del Producto

GammaLab 2.0 es una aplicación de escritorio standalone para análisis y visualización de señales neurofisiológicas (EEG), ejecutada localmente en estaciones de trabajo con Windows 10 o superior. El sistema mantiene una arquitectura modular orientada a plugins, donde un Kernel interno (IoC) orquesta servicios comunes como el almacenamiento de procesamientos, la gestión de eventos y la ejecución de tareas.

La aplicación no introduce dependencias de red ni bases de datos externas. GammaLab 2.0 pertenece a una familia de productos: es la segunda versión de GammaLab 1.0, desarrollada en la Pontificia Universidad Javeriana bajo la dirección de la misma directora y orientada al mismo cliente, el Laboratorio de Señales de la Universidad Nacional de Colombia.

Sobre la versión anterior, esta versión incorpora mejoras en la experiencia de usuario, persistencia y restauración de proyectos, una estrategia de paralelización del procesamiento implementada en el Kernel y un nuevo módulo de análisis basado en acoplamiento cruzado de frecuencias (*Cross-Frequency Coupling*).

El nuevo módulo incluye el análisis de acoplamiento fase-amplitud (*Phase-Amplitude Coupling*, PAC) sobre un canal, PAC entre dos canales y coherencia de fase (PPC). GammaLab 2.0 es compatible con los archivos y flujos de trabajo de su versión original.

### 2.1.1. Interfaces con el Sistema

GammaLab 2.0 no se integra con sistemas externos ni servicios en la nube. Su operación es completamente local y su contexto es el procesamiento de archivos de señales EEG adquiridos previamente con herramientas externas.

Las interfaces con el entorno son de dos tipos: interfaces de archivo para la entrada y salida de señales, e interfaces de usuario a través de la GUI, sin intercambio de mensajes con servicios remotos.

La comunicación entre los plugins y el Kernel del sistema se realiza a través del contrato de integración definido en la arquitectura interna. El Kernel incluye un servicio de tareas que ejecuta las operaciones pesadas en hilos trabajadores, fuera del hilo de la interfaz, de modo que esta se mantenga responsiva mientras se calcula.

### 2.1.2. Interfaces con el Usuario

Se describen los medios de interacción previstos y sus características técnicas:

- **Teclado.** Entrada de datos en formularios para la definición de parámetros como frecuencia de muestreo, rangos de frecuencias, selección de canales, selección de trials y

demás parámetros requeridos por los módulos de análisis.

- **Ratón/Trackpad.** Navegación y control de la GUI: selección de botones, menús, pestañas, canales, trials e interacción con gráficas.
- **Pantalla.** Visualización de la interfaz, pestañas y gráficos generados por cada módulo. Especificaciones: resolución mínima 1920 x 1080 px; tamaño recomendado mayor o igual a 14 pulgadas.
- **Interfaz gráfica (GUI).** Desarrollada en Python con PyQt5 y los componentes Qt. Está organizada en módulos navegables con barras laterales para la selección de la señal activa, la configuración de parámetros por plugin, la ejecución de análisis y la visualización de resultados. GammaLab 2.0 incorpora las vistas de PAC de un canal, PAC inter-canal y coherencia de fase.
- **Interfaz de entrada de archivos EEG.** Permite al usuario seleccionar y cargar archivos de señales adquiridos externamente en los formatos `.abf`, `.edf` y `.mat`, mediante el selector de archivos nativo de Windows.

### 2.1.3. Interfaces con el Hardware

GammaLab 2.0 no establece comunicación directa con equipos de adquisición de señales ni utiliza protocolos de red o puertos de comunicación, dado que opera exclusivamente de manera local sobre archivos previamente adquiridos.

La interacción con hardware se limita a dispositivos estándar de entrada, como teclado, ratón y trackpad, y de salida, como la pantalla. Durante la ejecución, los datos del sistema se almacenan temporalmente en memoria RAM y las tareas de procesamiento pueden distribuirse entre varios núcleos.

### 2.1.4. Interfaces con el Software

- **Windows.** Es el sistema operativo base sobre el que se ejecuta GammaLab 2.0. Se requiere Windows 10 de 64 bits o superior [1].
- **Python.** Es el lenguaje de programación base del sistema. Se utiliza la versión 3.11–3.12 [2].
- **PyQt5.** Es el framework sobre el que se construye la interfaz gráfica del sistema, manejando widgets, layouts, navegación y eventos mediante los componentes Qt [3].
- **VTK.** Es la librería utilizada para el renderizado embebido de señales EEG y la visualización científica 2D y 3D dentro de la aplicación [4].
- **NumPy.** Provee las estructuras de arreglos multidimensionales y las operaciones matemáticas base del sistema, incluyendo las operaciones requeridas por los análisis espectrales y el análisis de acoplamiento cruzado de frecuencias [5].

- **SciPy.** Complementa a NumPy con módulos especializados para procesamiento de señales, incluyendo el cálculo de densidad espectral de potencia, interpolaciones y filtros como Butterworth [6].
- **Pandas.** Gestiona la organización y exportación de resultados en formatos tabulares como `.csv`, `.xlsx` y `.json` [7].
- **PyWavelets.** Provee los algoritmos para el cálculo de la transformada wavelet y su promedio en el dominio tiempo-frecuencia [8].
- **pyABF.** Permite la lectura de archivos de electrofisiología en formato `.abf` [9].
- **pyEDFlib.** Permite la lectura de archivos de electrofisiología en formato `.edf` [10].
- **threading.** Módulo de la biblioteca estándar de Python utilizado por el servicio de tareas del Kernel para coordinar los hilos de ejecución, en particular la señal de cancelación cooperativa entre la interfaz y el hilo trabajador [11].
- **QThread (Qt).** Clase de Qt, disponible a través de PyQt5, que implementa los hilos trabajadores del servicio de tareas. Las operaciones pesadas que se despachan a este servicio (por ejemplo, la carga de señales, Wavelet, PAC y el índice de modulación) se ejecutan en estos hilos, fuera del hilo de la interfaz, y notifican su progreso y su resultado mediante señales de Qt [12].
- **pytest.** Es el framework utilizado para la validación interna del sistema mediante pruebas unitarias, cubriendo módulos de carga de señales, filtrado, generación de trials, análisis PAC, análisis inter-canal, exportación y ejecución de tareas en segundo plano [14].

### 2.1.5. Interfaces de Comunicación

GammaLab 2.0 no utiliza interfaces de comunicación de red. El sistema no implementa protocolos como TCP/IP, HTTP, sockets ni buses de mensajería, y no requiere conexión a redes locales o remotas para su operación.

Todo el procesamiento ocurre localmente en el equipo del usuario, sin intercambio de datos con sistemas externos.

### 2.1.6. Restricciones de Memoria

- **Entorno Python.** GammaLab 2.0 se desarrolla en Python 3.11+, requiriendo espacio en disco para el entorno base y sus dependencias.
- **Librerías Científicas.** El uso de NumPy, SciPy, Pandas y PyWavelets genera un consumo adicional de memoria durante el procesamiento de señales EEG, dependiendo del tamaño de los arreglos manipulados y del número de trials seleccionados.

- **Interfaz Gráfica.** La GUI construida con PyQt5 demanda memoria adicional durante la ejecución, especialmente cuando se visualizan simultáneamente varias gráficas y resultados.
- **Visualización Científica.** Las operaciones de renderizado mediante VTK pueden incrementar el consumo de RAM y VRAM dependiendo de la cantidad de señales y resultados visualizados.
- **Lectura de Archivos EEG.** Las librerías pyEDFlib y pyABF presentan un consumo variable según la cantidad de canales y puntos de cada señal.
- **Ejecución de tareas en segundo plano.** El servicio de tareas del Kernel ejecuta las operaciones pesadas en hilos trabajadores (*QThread*) distintos del hilo de la interfaz. Los hilos comparten el espacio de memoria del proceso, por lo que no se duplica el entorno de ejecución: el consumo adicional depende de los arreglos que cada tarea recibe y de los resultados que produce. Las tareas se atienden de una en una, en cola; una tarea cancelada que no atiende la cancelación conserva su hilo y su memoria hasta terminar.
- **Reparto del trabajo entre hilos.** Los análisis que procesan varios trials de forma independiente (por ejemplo, PAC Average) reparten los trials entre varios hilos dentro de la misma tarea. El número de hilos se decide en el equipo donde corre el sistema: nunca supera cuatro, la mitad de los núcleos disponibles ni lo que cabe en la memoria (como máximo un cuarto de la RAM libre, con un tope de 3 GB). Cada hilo en vuelo reserva del orden de cientos de megabytes, según la longitud del trial; en un equipo con pocos núcleos o poca memoria el sistema usa un solo hilo.

Con base en lo anterior, se definen los siguientes requerimientos de hardware:

#### Requerimientos mínimos

- **Procesador:** CPU de 4 núcleos a 2.0 GHz o superior (AMD Ryzen 3 / Intel i3).
- **RAM:** 8 GB.
- **VRAM:** 1 GB (GPU integrada).
- **Almacenamiento:** 3 GB libres.

#### Requerimientos recomendados

- **Procesador:** CPU de 4–8 núcleos a 3.0 GHz o superior (AMD Ryzen 5 / Intel i5 o superior).
- **RAM:** 16 GB.
- **VRAM:** 2 GB.
- **Almacenamiento:** 20 GB libres en SSD.

### 2.1.7. Operaciones

Durante su uso, GammaLab 2.0 permite al usuario realizar las siguientes operaciones principales:

- **Cargar señal.** Seleccionar y cargar archivos de señal en formatos `.abf`, `.edf` y `.mat`.
- **Preprocesamiento.** Dividir la señal en trials, eliminar artefactos de estimulación y aplicar filtros como Butterworth.
- **Análisis.** Procesar la señal en los dominios de tiempo, frecuencia y tiempo-frecuencia, incluyendo FFT, PSD, Wavelet, ERP, análisis PAC de un canal, PAC inter-canal y coherencia de fase PPC.
- **Análisis PAC.** Seleccionar bandas de fase y amplitud, seleccionar un trial o un promedio de trials, calcular el acoplamiento fase-amplitud y visualizar sus resultados.
- **Análisis inter-canal.** Seleccionar un canal como fuente de fase y otro como fuente de amplitud para calcular PAC inter-canal o coherencia de fase PPC.
- **Visualización y exportación.** Visualizar los resultados del procesamiento y exportarlos en formatos numéricos y gráficos como `.csv`, `.xlsx`, `.json`, `.jpeg`, `.jpg`, `.png` y `.tiff`.
- **Cálculo de medidas.** Calcular pendiente o amplitud sobre procesamientos realizados.
- **Ayuda.** Acceder al manual de usuario, documentación, tutoriales y última versión estable.

En GammaLab 2.0 se incorpora adicionalmente un instalador ejecutable, de modo que el usuario puede iniciar la aplicación directamente desde un ícono de escritorio sin requerir configuración manual del entorno Python.

### 2.1.8. Periodos de Actividad e Inactividad

GammaLab 2.0 ejecuta sus funciones mientras la aplicación está abierta. No mantiene procesos en segundo plano ni mecanismos de sincronización con servicios externos.

El sistema se considera activo cuando la aplicación está abierta y ejecuta operaciones de carga, análisis, renderizado, exportación o cálculo. Durante este periodo consume CPU, RAM y GPU según el volumen de las señales procesadas.

El sistema se considera inactivo cuando la aplicación ha sido cerrada manualmente o el sistema operativo ha detenido su ejecución. En este estado no consume CPU ni memoria y los proyectos permanecen almacenados en disco.

### 2.1.9. Tratamiento de Sesiones y Persistencia

GammaLab 2.0 permite guardar y restaurar proyectos de trabajo. Una sesión puede incluir archivos cargados, datasets activos, trials generados, parámetros configurados, análisis ejecutados, resultados almacenados y ventanas activas.

El usuario puede guardar manualmente el proyecto mediante las opciones *Save* y *Save As*, y abrirlo posteriormente mediante la opción *Open Project*.

Los parámetros utilizados en cada ejecución del análisis PAC se almacenan junto con sus resultados. La persistencia del proyecto es independiente de la exportación de resultados, por lo que los datos numéricos y las representaciones gráficas también pueden exportarse manualmente.

### 2.1.10. Funciones de Soporte al Procesamiento

GammaLab 2.0 utiliza las siguientes librerías para el procesamiento, la visualización de señales y la ejecución de tareas en segundo plano:

- **NumPy y SciPy:** operaciones matemáticas, filtrado y transformadas.
- **Cálculo propio de PAC y del índice de modulación:** implementado sobre NumPy y SciPy a partir de los algoritmos de referencia en MATLAB, sin una librería externa dedicada.
- **Pandas:** manipulación estructurada y exportación de datos.
- **PyWavelets:** análisis tiempo-frecuencia mediante transformadas wavelet.
- **VTK:** renderizado y visualización 2D y 3D.
- **pyEDFlib y pyABF:** lectura de formatos estándar de adquisición neurofisiológica.
- **threading y QThread (PyQt5):** hilos de ejecución del servicio de tareas del Kernel: ejecución en segundo plano, cola de tareas, cancelación cooperativa (`threading.Event`) y señales de progreso hacia la interfaz.

### 2.1.11. Requerimientos de Adaptación del Sitio

GammaLab 2.0 está diseñado para ejecutarse localmente en equipos con Windows 10 o superior. Para su correcta instalación y operación, el equipo debe cumplir las siguientes condiciones:

- Permisos de lectura y escritura en la carpeta de instalación de la aplicación.
- Permisos para ejecutar scripts por consola en Windows.
- Python 3.11–3.13 instalado a nivel local, con disponibilidad para instalar las dependencias requeridas.

- Políticas de TI que permitan la ejecución de aplicaciones locales sin necesidad de acceso a red.
- Disponibilidad de espacio en disco y memoria RAM según los requerimientos definidos en la sección correspondiente.

## 2.2. Funciones del Producto

GammaLab 2.0 mantiene la totalidad de las funcionalidades de GammaLab 1.0, organizadas por ventanas según la arquitectura modular del sistema. Sobre esta base, la segunda versión introduce correcciones a los flujos de navegación existentes, mejoras en la experiencia de usuario, persistencia y restauración de proyectos, y un nuevo módulo de análisis basado en acoplamiento cruzado de frecuencias.

El nuevo módulo incluye PAC de un canal, PAC inter-canal y coherencia de fase PPC. Adicionalmente, la ejecución de los plugins puede realizarse en paralelo desde el Kernel, reduciendo los tiempos de respuesta sin modificar el comportamiento visible de cada módulo para el usuario.

### 2.2.1. Pantalla general

La pantalla general de la aplicación está compuesta por una barra principal de funciones, la cual contiene las opciones *File*, *Home*, *Preprocessing* y *Analysis*.

En la barra lateral izquierda se encuentran tres iconos con las siguientes funciones:

- El primero permite visualizar la carpeta en la que se encuentra almacenado el proyecto.
- El segundo despliega una ventana inferior en la que se presentan los resultados de *slope* y *amplitude*.
- El tercero permite consultar el manual de usuario.

En la pestaña *File* se encuentran las siguientes opciones:

- *Open Signal*: permite cargar una señal individual dentro de la aplicación.
- *Open Project*: permite abrir un proyecto previamente guardado.
- *Save*: guarda los cambios realizados sobre el proyecto actual.
- *Save As*: permite guardar el proyecto actual bajo un nuevo nombre o ubicación.
- *Close Project*: cierra el proyecto que se encuentra abierto.
- *Exit*: cierra la aplicación.

### 2.2.2. Home

En la pestaña *Home* se encuentran tres categorías principales dentro de la barra de funcionalidades:

- **Measurements**: contiene las siguientes opciones:
  - *Slope*, con las subopciones *Slope 2 Trials* y *Slope All Trials*.
  - *Amplitude*.
  - Eliminar la última medición o la totalidad de las mediciones.
  - Mostrar u ocultar las mediciones en pantalla.
- **Zoom**: permite activar o desactivar la función de acercamiento sobre la señal.

### 2.2.3. Preprocessing

Agrupa los módulos de preparación de la señal para su análisis posterior.

- **Trials.** Divide la señal en ensayos, permitiendo seleccionar el canal a dividir, el canal de estímulos, el umbral (*threshold*), el número de estímulos y el tamaño de ventana. El usuario puede navegar entre los trials generados e incluirlos o descartarlos.
- **Remove Artifact.** Elimina artefactos presentes en los trials, permitiendo seleccionar el rango de tiempo a eliminar y escoger entre dos modos de corrección: interpolación del segmento o eliminación desde el inicio hasta el punto seleccionado.
- **Filter.** Aplica filtros a la señal, con selección del tipo de filtro (Butterworth), el rango de frecuencias, el orden y visualización del resultado sobre la señal completa y el primer trial.

### 2.2.4. Analysis

Agrupa los módulos de análisis en los dominios de tiempo, frecuencia y tiempo-frecuencia.

- **Average.** Calcula el promedio temporal de los trials generados y lo visualiza.
- **Event Related Potential (ERP).** Calcula el potencial relacionado a eventos, generando un *Butterfly Plot* con todos los trials alineados y un espectrograma de dinámica tiempo-frecuencia.
- **FFT.** Realiza la Transformada de Fourier, con definición de frecuencia de muestreo, rango de frecuencias y visualización del espectro de potencia.
- **FFT Average.** Calcula el promedio del FFT sobre los trials, con selección de frecuencia de muestreo y rango de frecuencias de interés.

- **PSD.** Calcula la Densidad Espectral de Potencia mediante el método de Welch, con selección de modo de cálculo, tipo de ventana, rango de frecuencias y modo de *detrend*.
- **PSD Average.** Promedia múltiples PSD de forma agregada, con configuración de frecuencia de muestreo, ventana y rango de frecuencias.
- **PSD Relative.** Calcula la PSD relativa, entregando potencia absoluta y relativa por banda, con selección de ventana, *detrend* y modo de cálculo.
- **Wavelet.** Calcula la Transformada Wavelet con configuración de frecuencia de muestreo, rango de frecuencias, número de ciclos, normalización y escala de visualización.
- **Wavelet Average.** Calcula el promedio de la Transformada Wavelet sobre múltiples trials, generando un mapa tiempo-frecuencia agrupado.
- **Phase-Amplitude Coupling (PAC) de un canal.** Calcula la relación entre la fase de una banda de baja frecuencia y la amplitud de un rango de frecuencias de una señal EEG mediante transformada wavelet. El usuario puede seleccionar la banda de fase, el rango de frecuencias de amplitud y el modo de trial, que puede corresponder a un trial individual o al promedio de varios trials.

  El resultado incluye un mapa de acoplamiento fase-frecuencia, un histograma de distribución de amplitud en función de la fase y un vector de acoplamiento medio con su magnitud y ángulo.
- **PAC inter-canal.** Permite seleccionar un canal como fuente de fase y otro como fuente de amplitud para calcular el acoplamiento fase-amplitud entre ambos canales. El resultado incluye el mapa de acoplamiento fase-frecuencia, el histograma de fase y el vector de acoplamiento medio.
- **Coherencia de fase (PPC).** Permite seleccionar dos canales y calcular la coherencia de fase entre ellos. El resultado puede visualizarse y exportarse junto con sus datos numéricos.

Para ejecutar el análisis PAC debe existir una señal cargada con al menos un trial válido. Para ejecutar PAC inter-canal o PPC deben existir al menos dos canales con trials generados.

Las bandas de frecuencia seleccionadas deben encontrarse por debajo de la frecuencia de Nyquist. Además, el límite inferior de cada rango debe ser menor que el límite superior. Si los valores no cumplen estas condiciones, el sistema muestra un mensaje de error y solicita nuevos valores.

## 2.3. Características del Usuario

La herramienta está dirigida a usuarios interesados en el análisis y procesamiento de señales EEG, como estudiantes, investigadores y personal técnico. En el contexto de este proyecto, el cliente objetivo corresponde al Laboratorio de Electrofisiología de la Universidad Nacional de Colombia, sede Bogotá.

| Características del usuario | Descripción |
|---|---|
| **Nivel de Seguridad o de Privilegios** | GammaLab 2.0 no define niveles de seguridad diferenciados, por lo que el usuario tiene acceso a todas las funcionalidades disponibles, incluyendo carga de señales, preprocesamiento, ejecución de análisis, visualización, persistencia y exportación de resultados. |
| **Roles** | El usuario es responsable de realizar tareas relacionadas con el procesamiento y análisis de señales EEG, incluyendo la preparación de señales, configuración de parámetros, ejecución de algoritmos y análisis de resultados obtenidos. |
| **Nivel de Estudios o Experiencia Técnica** | Se espera que el usuario posea conocimientos básicos o intermedios en adquisición y análisis de señales EEG, incluyendo conceptos como frecuencia de muestreo, segmentación en trials, filtrado, eliminación de artefactos y análisis en dominios de tiempo, frecuencia y tiempo-frecuencia. También se requiere familiaridad con comodulogramas, histogramas de fase y resultados de acoplamiento entre frecuencias. |
| **Frecuencia de Uso** | El sistema está orientado a usuarios que realizan actividades recurrentes de análisis y procesamiento de señales neurofisiológicas en entornos académicos, investigativos o de laboratorio. |

Tabla 3: Características generales de los usuarios de GammaLab 2.0

## 2.4. Restricciones

Esta sección define los límites técnicos, normativos y operativos que condicionan el diseño, implementación y despliegue de GammaLab 2.0.

### 2.4.1. Restricciones Generales

El sistema opera íntegramente en inglés, tanto en la interfaz como en mensajes de error y alertas. Ante fallos internos o errores del usuario, el sistema debe mostrar mensajes claros sin interrumpir el flujo de ejecución.

El sistema debe soportar la carga de archivos con múltiples canales y permitir realizar análisis sobre al menos uno de ellos. Para los análisis inter-canal debe permitir seleccionar dos

canales. Las gráficas, tablas y resultados numéricos generados deben poder exportarse desde la interfaz.

### 2.4.2. Restricciones de Software

El sistema mantiene una arquitectura modular orientada a plugins, donde el Kernel orquesta la ejecución y permite incorporar nuevas capacidades sin modificar los plugins existentes.

El sistema se desarrolla en Python 3.11–3.12 y se distribuye como aplicación ejecutable localmente en Windows; alternativamente puede ejecutarse directamente desde el código fuente del repositorio oficial.

El procesamiento de señales utiliza NumPy, SciPy y PyWavelets. La interfaz gráfica se construye con PyQt5 y los componentes Qt. Los formatos de entrada soportados son `.abf`, `.edf` y `.mat`.

La ejecución de tareas en segundo plano del Kernel se implementa con hilos (*QThread* de Qt y el módulo `threading` de la biblioteca estándar de Python).

### 2.4.3. Restricciones de Hardware

El sistema debe ejecutarse en Windows 10 de 64 bits o superior. Los requerimientos mínimos de hardware son:

- CPU de 4 núcleos a 2.0 GHz.
- 8 GB de RAM.
- 1 GB de VRAM.
- 3 GB de almacenamiento libre.

## 2.5. Modelo del dominio

El modelo del dominio describe las entidades conceptuales fundamentales involucradas en el funcionamiento de Gamma Lab 2.0 y las relaciones existentes entre ellas.

Estas entidades representan la estructura lógica utilizada por el sistema para gestionar señales EEG, ensayos experimentales, resultados derivados de análisis neurofisiológicos y sesiones persistentes de trabajo.

Gamma Lab 2.0 extiende la arquitectura desarrollada en Gamma Lab 1.0 mediante la incorporación de capacidades avanzadas relacionadas con:

- Persistencia y restauración automática de proyectos.
- Procesamiento paralelo de señales EEG.

- Gestión centralizada de datasets experimentales.
- Exportación y almacenamiento de resultados analíticos.
- Análisis de acoplamiento cruzado de frecuencias (*Cross-Frequency Coupling*).

El modelo propuesto permite desacoplar la lógica de procesamiento, persistencia y visualización, facilitando la extensibilidad, mantenibilidad y escalabilidad de la plataforma.

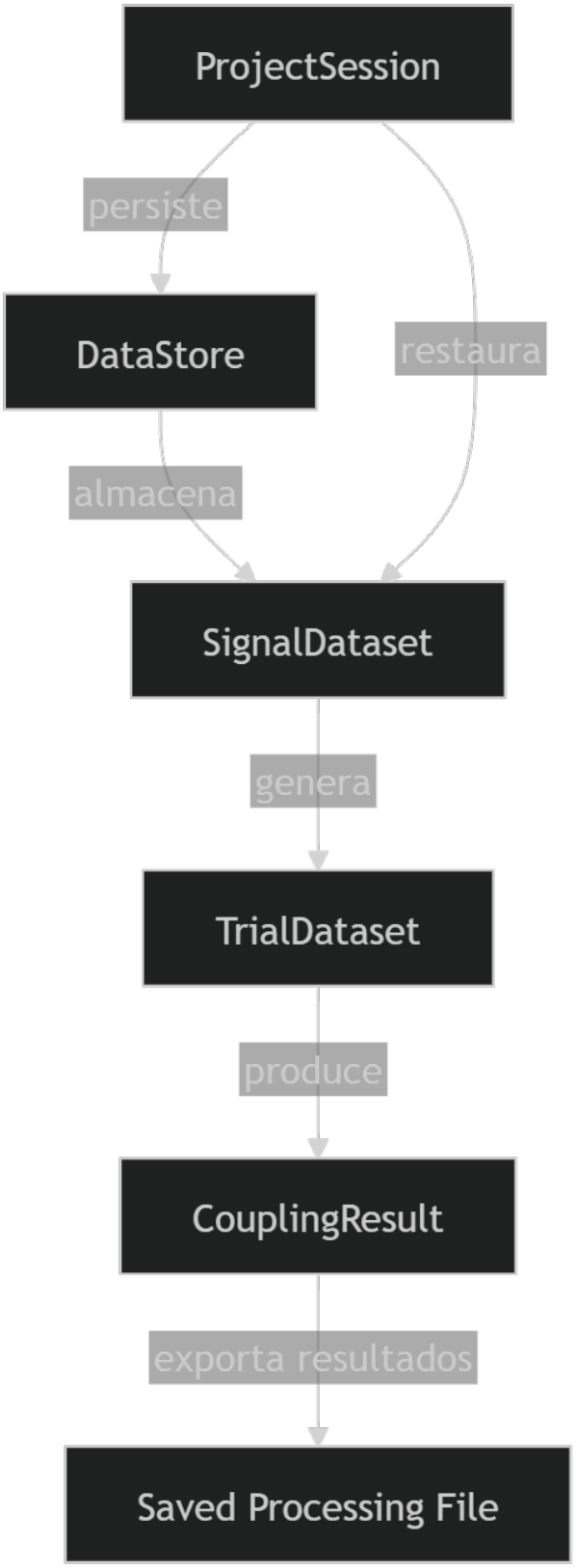



Figura 1: Modelo conceptual del dominio de Gamma Lab 2.0. Las entidades representan los componentes centrales de almacenamiento, procesamiento y persistencia de datos neurofisiológicos.

### 2.5.1. DataStore

| | |
|---|---|
| **ID** | 1 |
| **Elemento** | DataStore |
| **Descripción** | Entidad responsable de gestionar el almacenamiento interno de señales, datasets experimentales, resultados analíticos y estructuras de sesión generadas durante la ejecución del sistema. Centraliza la persistencia temporal y permanente de la información utilizada por los diferentes módulos de Gamma Lab 2.0. |
| **Responsabilidades** | Administrar el acceso a datos, mantener referencias a datasets activos, coordinar operaciones de lectura y escritura y facilitar la persistencia de resultados de análisis neurofisiológicos. |
| **Objetivo** | Proporcionar una capa centralizada de administración de datos que reduzca el acoplamiento entre módulos y facilite la reutilización, persistencia y recuperación de información experimental. |

Tabla 4: Entidad de dominio: DataStore

### 2.5.2. SignalDataset

| | |
|---|---|
| **ID** | 2 |
| **Elemento** | SignalDataset |
| **Descripción** | Representa una señal EEG cargada dentro del sistema junto con la estructura de información requerida para su procesamiento y análisis. Incluye datos relacionados con frecuencia de muestreo, canales, metadatos experimentales y formato del archivo original. |
| **Atributos principales** | Nombre de la señal, formato del archivo, frecuencia de muestreo, canales registrados, unidades físicas, duración temporal y metadatos experimentales asociados. |
| **Objetivo** | Estandarizar la representación interna de señales neurofisiológicas para garantizar interoperabilidad entre los módulos de visualización, preprocesamiento y análisis. |

Tabla 5: Entidad de dominio: SignalDataset

### 2.5.3. TrialDataset

| | |
|---|---|
| **ID** | 3 |
| **Elemento** | TrialDataset |
| **Descripción** | Representa el conjunto estructurado de ensayos (*trials*) derivados de una señal EEG. Cada ensayo corresponde a un segmento temporal asociado a un estímulo, marcador o evento experimental específico. |
| **Atributos principales** | Canal asociado, frecuencia de muestreo, límites temporales, vector temporal relativo, matriz de ensayos y parámetros experimentales derivados. |
| **Objetivo** | Permitir el procesamiento organizado y eficiente de ensayos experimentales para análisis temporales, espectrales, tiempo-frecuencia y análisis de acoplamiento entre frecuencias. |

Tabla 6: Entidad de dominio: TrialDataset

### 2.5.4. CouplingResult

| | |
|---|---|
| **ID** | 4 |
| **Elemento** | CouplingResult |
| **Descripción** | Entidad encargada de almacenar los resultados generados por el módulo de análisis de acoplamiento cruzado de frecuencias (*Cross-Frequency Coupling*). Incluye resultados de PAC de un canal, PAC inter-canal y coherencia de fase PPC. |
| **Atributos principales** | Bandas de frecuencia de fase y amplitud, canales utilizados, trials seleccionados, tipo de análisis, índice de modulación, histograma de fase, magnitud y ángulo del vector de acoplamiento, matriz del comodulograma, parámetros de exportación y referencias a archivos generados. |
| **Responsabilidades** | Gestionar resultados PAC y PPC, almacenar métricas derivadas y parámetros utilizados, facilitar la exportación de análisis y permitir la persistencia de archivos de salida generados durante el procesamiento de señales neurofisiológicas. |
| **Objetivo** | Persistir resultados analíticos y artefactos derivados del procesamiento EEG para su visualización, exportación, comparación experimental y reutilización en sesiones futuras. |

Tabla 7: Entidad de dominio: CouplingResult

### 2.5.5. ProjectSession

| | |
|---|---|
| **ID** | 5 |
| **Elemento** | ProjectSession |
| **Descripción** | Representa el estado persistente de una sesión de trabajo dentro de Gamma Lab 2.0, incluyendo archivos cargados, parámetros configurados, análisis ejecutados, resultados almacenados y ventanas activas. |
| **Atributos principales** | Identificador del proyecto, fecha de creación, datasets activos, configuraciones de análisis, historial de procesamiento y ventanas abiertas. |
| **Objetivo** | Permitir la restauración automática de sesiones y la gestión estructurada de proyectos experimentales dentro del sistema. |

Tabla 8: Entidad de dominio: ProjectSession

## 2.6. Suposiciones y dependencias

La presente sección describe las condiciones asumidas durante el desarrollo de Gamma Lab 2.0 y las dependencias externas necesarias para garantizar el funcionamiento adecuado del sistema.

### 2.6.1. Suposiciones técnicas

- El sistema será ejecutado principalmente sobre sistemas operativos Windows 10 o superiores en arquitectura de 64 bits.
- El usuario dispondrá de permisos de lectura y escritura sobre el sistema de archivos local.
- Los archivos EEG utilizados estarán en formatos compatibles soportados por la plataforma (`.abf`, `.edf` y `.mat`).
- Las señales utilizadas para el análisis PAC tendrán al menos un trial válido.
- Las señales utilizadas para PAC inter-canal y PPC tendrán al menos dos canales con trials generados.
- Las bandas de frecuencia seleccionadas para el análisis se encontrarán por debajo de la frecuencia de Nyquist.
- El entorno Python contará con las dependencias científicas requeridas por los módulos de procesamiento.

- El equipo de ejecución dispondrá de recursos computacionales suficientes para manipular señales EEG de gran tamaño.
- Los módulos desarrollados por terceros respetarán la arquitectura modular y las interfaces definidas por Gamma Lab.

### 2.6.2. Suposiciones sobre el usuario

- El usuario posee conocimientos básicos o intermedios sobre análisis de señales EEG y neurofisiología computacional.
- El usuario comprende conceptos relacionados con análisis temporal, frecuencial y tiempo-frecuencia.
- El usuario tiene experiencia básica utilizando interfaces gráficas de escritorio orientadas a análisis científico.
- El usuario es capaz de interpretar gráficas, espectros, comodulogramas, histogramas de fase y resultados estadísticos derivados del sistema.
- El usuario posee conocimientos básicos de inglés técnico para comprender terminología científica y nombres de librerías.

### 2.6.3. Dependencias externas y del sistema

- El sistema depende de bibliotecas científicas externas como NumPy, SciPy, PyWavelets, PyEDFlib, PyABF y Pandas.
- La interfaz gráfica depende del framework PyQt5 y de los componentes Qt.
- El almacenamiento y control de versiones del proyecto depende de Git y GitHub.
- La exportación de resultados depende de librerías de visualización y generación de archivos CSV, Excel, JSON e imágenes.
- El módulo de análisis PAC depende de las capacidades de procesamiento de señales proporcionadas por NumPy y SciPy.
- El servicio de tareas del Kernel depende de los hilos de Qt (*QThread*) y del módulo `threading` del entorno Python.

# 3. REQUERIMIENTOS ESPECÍFICOS

Este capítulo define los requerimientos específicos de GammaLab 2.0, derivados de las necesidades y brechas identificadas en la primera versión de la herramienta. Los requerimientos se dividen entre funcionales y no funcionales, y establecen las condiciones que deberá cumplir la nueva iteración para materializar la solución propuesta y orientar su posterior implementación y validación.

## 3.1. Nomenclatura de Requerimientos

Con el fin de facilitar la trazabilidad y la referencia cruzada entre los distintos documentos que soportan este trabajo, se definió una nomenclatura que distingue los requisitos funcionales de los no funcionales.

Los requerimientos funcionales se identifican mediante el prefijo FR-XX-YY, donde XX corresponde al área funcional de Gamma Lab 2.0 a la que se encuentra asociado el requisito y YY corresponde al número consecutivo dentro de dicha área. Esta clasificación busca mantener una correspondencia entre los requisitos y las principales capacidades funcionales involucradas en la evolución de la herramienta, facilitando su seguimiento a lo largo de las diferentes etapas de diseño, desarrollo y validación.

A partir de esta clasificación, los requerimientos funcionales se organizan en cuatro áreas:

- **IO**: Entrada/Salida: agrupa los requerimientos relacionados con la carga y persistencia de señales, así como el guardado y la apertura de proyectos, consolidando el manejo de datos como base funcional de la herramienta.
- **UI**: Interacción general: agrupa los requerimientos orientados a mejorar la experiencia de usuario, la edición de la señal y la gestión de trials.
- **WV**: Wavelet/Escalograma: agrupa los requerimientos que expanden y ajustan las capacidades de esta representación tiempo-frecuencia.
- **PAC**: Acoplamiento fase-amplitud: agrupa los requerimientos del nuevo módulo de análisis incorporado en esta iteración, e incluye tanto el acoplamiento intra-canal como el inter-canal y la coherencia de fase.

Los requerimientos no funcionales se identifican mediante el prefijo **NFR-X-YY**, donde $X$ corresponde a la categoría del modelo FURPS+ [16] y $YY$ es el número consecutivo dentro de dicha categoría. Estos requisitos concentran, en particular, los lineamientos orientados a la estabilización y el fortalecimiento arquitectónico de la herramienta. Se identificaron requisitos en cuatro de las categorías del modelo:

- **U**: Usabilidad
- **P**: Rendimiento (Performance)
- **R**: Confiabilidad (Reliability)

- **S**: Soporte (Supportability)

## 3.2. Requerimientos Funcionales

La definición de los requerimientos funcionales partió del levantamiento de necesidades con el laboratorio y de la evaluación de Gamma Lab 1.0 durante su uso. Este ejercicio permitió identificar brechas entre las capacidades de la herramienta y las necesidades del trabajo experimental, así como dificultades que comprometían su uso autónomo por parte de los investigadores. La interacción resultaba poco intuitiva y dificultaba mantener el contexto de trabajo durante la navegación. De hecho, ningún usuario logró navegar por la herramienta sin asistencia durante las pruebas realizadas. Además, la falta de información sobre el estado de las operaciones generaba incertidumbre cuando la interfaz dejaba de responder durante la carga o el procesamiento de datos. Estas situaciones dificultan la construcción de un modelo mental claro del funcionamiento de la herramienta para el usuario.

También se identificaron limitaciones en la continuidad del trabajo y el aprovechamiento de los resultados. La gestión de las señales y los ensayos restringía la flexibilidad de los procedimientos analíticos, mientras que la ausencia de persistencia dificultaba retomar el trabajo entre sesiones. A esto se sumaban restricciones en el intercambio de datos y en las capacidades de análisis disponibles, que no cubrían completamente las necesidades investigativas del laboratorio. En conjunto, estas brechas afectaban la fluidez del trabajo experimental y evidenciaban la necesidad de fortalecer tanto la interacción con la herramienta como el manejo de la información y el alcance de los análisis.

A partir de este diagnóstico se definieron los requerimientos funcionales de Gamma Lab 2.0, mediante la traducción de las dificultades observadas y las necesidades expresadas por los investigadores en especificaciones sobre el comportamiento esperado del sistema. Las tablas siguientes presentan estos requerimientos de manera individual, agrupados por los modulos definidos en la sección de nomenclatura 3.1.

### 3.2.1. Módulo IO: Entrada/Salida

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-IO-01 | R08 | El sistema debe persistir la señal cargada |
| FR-IO-02 | R80 | El sistema debe cargar señales en los formatos ABF, EDF y MAT. |
| FR-IO-03 | R07 | El sistema debe poder guardar un proyecto en un archivo con la extensión .glab que registre el nombre y la ruta del archivo de señal de origen junto con el trabajo realizado sobre ella, sin incluir los datos de la señal dentro de ese archivo. |

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-IO-04 | R78 | El sistema debe abrir un proyecto guardado, restaurando la señal referenciada y el trabajo registrado en él, e informar cuando el archivo de señal referenciado ya no esté disponible. |
| FR-IO-05 | R05 | El sistema debe exportar los resultados de la sección de análisis en los formatos Excel, CSV, JSON, JPEG, JPG, PNG y TIFF. |
| FR-IO-06 | R09 | El sistema debe exportar a CSV una única columna de eje X cuando todas las series comparten el mismo eje. |
| FR-IO-07 | R28 | El sistema debe asignar un nombre de exportación en el que se incluya el nombre de la señal |

Tabla 10: Requerimientos funcionales del módulo IO

### 3.2.2. Módulo UI: Interacción general

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-UI-01 | R01 | El sistema debe permitir realizar cortes de la señal |
| FR-UI-02 | R24 | El sistema debe permitir modificar un conjunto de trials ya creado, regenerándolo con otros parámetros, descartando trials o aplicándole preprocesamiento, sin alterar el conjunto original. |
| FR-UI-03 | R25 | El sistema debe restablecer a sus valores por defecto todos los parámetros de una funcionalidad mediante un botón Clear, presente en cada funcionalidad que se ejecute con un botón. |
| FR-UI-04 | R26 | El sistema debe mostrar los valores de los ejes X y Y correspondientes al punto de la señal sobre el que se posiciona el cursor. |
| FR-UI-05 | R27 | El sistema debe permitir escoger qué trials participan en cada análisis y descartar los que no deban usarse, conservándolos en el conjunto. |
| FR-UI-06 | R29 | El sistema debe ajustar el tiempo de muestreo |
| FR-UI-07 | R30 | El sistema debe tener zoom en todas las direcciones |
| FR-UI-08 | R31 | El sistema debe mostrar el umbral en mV |
| FR-UI-09 | R32 | El sistema debe mostrar Butterfly explicitamente |
| FR-UI-10 | R36 | El sistema debe realizar el promedio en todos los trials |

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-UI-11 | R37 | El sistema debe indicar en todo momento en qué trial se está trabajando. |
| FR-UI-12 | R38 | El sistema debe indicar qué trials participan en cada operación. |
| FR-UI-13 | R40 | El sistema debe remover un artefacto con medidas |
| FR-UI-14 | R41 | El sistema debe promediar los trials específicos que el usuario seleccione. |
| FR-UI-15 | R68 | El sistema debe identificar cada conjunto de trials por la pareja (archivo, canal) y devolver los trials del canal solicitado, sin reemplazarlo por el último generado. |

Tabla 12: Requerimientos funcionales del módulo UI

### 3.2.3. Módulo WV: Wavelet/Escalograma

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-WV-01 | R33 | El sistema debe permitir elegir el tipo de escala del escalograma de wavelet y ajustar el rango de valores representado. |
| FR-WV-02 | R34 | El sistema debe normalizar el escalograma de wavelet según el método que elija el usuario. |
| FR-WV-03 | R35 | El sistema debe exportar wavelet en EMF |

Tabla 14: Requerimientos funcionales del módulo WV

### 3.2.4. Módulo PAC: Acoplamiento fase-amplitud

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-PAC-01 | R13 | El sistema debe ejecutar el análisis de acoplamiento fase-amplitud (PAC) sobre una señal electrofisiológica cargada en el sistema. |
| FR-PAC-02 | R14 | El sistema debe seleccionar la banda de frecuencia correspondiente a la fase para el análisis PAC. |
| FR-PAC-03 | R15 | El sistema debe seleccionar la banda de frecuencia correspondiente a la amplitud para el análisis PAC. |
| FR-PAC-04 | R16 | El sistema debe permitir definir el intervalo temporal (o los trials) sobre el cual se ejecutará el análisis PAC. |

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-PAC-05 | R17 | El sistema debe calcular, para la banda de fase seleccionada y el rango de amplitud seleccionado, un mapa de acoplamiento fase-frecuencia mediante transformada |
| FR-PAC-06 | R18 | El sistema debe representar los resultados del análisis PAC mediante un mapa fase-frecuencia (fase en el eje X, frecuencia de amplitud en el eje Y). |
| FR-PAC-07 | R19 | El sistema debe representar la distribución de amplitud en función de la fase (histograma de fase) obtenida durante el análisis PAC. |
| FR-PAC-08 | R20 | El sistema debe calcular y mostrar el valor del vector de acoplamiento medio (magnitud y ángulo) derivado del histograma de fase. |
| FR-PAC-09 | R21 | El sistema debe exportar los resultados numéricos del análisis PAC de un canal. |
| FR-PAC-10 | R22 | El sistema debe exportar las representaciones gráficas generadas durante el análisis PAC de un canal. |
| FR-PAC-11 | R23 | El sistema debe almacenar los parámetros utilizados en cada ejecución del análisis PAC junto con sus resultados. |
| FR-PAC-12 | R60 | El sistema debe permitir seleccionar dos canales de la misma señal: uno como fuente de fase y otro como fuente de amplitud. |
| FR-PAC-13 | R61 | El sistema debe calcular el mapa de acoplamiento fase-frecuencia entre el canal de fase y el canal de amplitud seleccionados. |
| FR-PAC-14 | R62 | El sistema debe representar los resultados del análisis PAC inter-canal con las mismas visualizaciones que el PAC de un canal. |
| FR-PAC-15 | R63 | El sistema debe permitir seleccionar dos canales y un rango de frecuencias para calcular la coherencia de fase entre ellos. |
| FR-PAC-16 | R64 | El sistema debe representar la coherencia de fase mediante un mapa tiempo-frecuencia. |
| FR-PAC-17 | R65 | El sistema debe exportar los resultados numéricos del análisis PAC inter-canal y de coherencia de fase (PPC). |
| FR-PAC-18 | R66 | El sistema debe exportar las representaciones gráficas del análisis PAC inter-canal y de coherencia de fase (PPC) como imagen. |

| Código | N° Matriz | Especificación |
|---|---|---|
| FR-PAC-19 | R75 | El sistema debe segmentar la señal filtrada en la banda de frecuencia rápida tomando cada ciclo de la onda lenta como una época, y superponer todas las épocas en una misma gráfica de tipo butterfly con la fase del ciclo lento en el eje X. |
| FR-PAC-20 | R76 | El sistema debe aplicar sobre la señal activa un filtro Butterworth pasa-banda de fase cero, con frecuencia de corte inferior, frecuencia de corte superior y orden configurables por el usuario. |

Tabla 16: Requerimientos funcionales del módulo PAC

## 3.3. Requerimientos No Funcionales

La definición de los requerimientos no funcionales partió de una revisión técnica de Gamma Lab 1.0, orientada a comprender su arquitectura, la interacción entre sus componentes y el comportamiento de la aplicación durante la ejecución. El análisis evidenció que, aunque la herramienta cuenta con una estructura modular basada en complementos independientes, los mecanismos de coordinación entre estos son insuficientes para gestionar adecuadamente sus dependencias y su ciclo de vida. Asimismo, la gestión del estado compartido y de los recursos no garantiza un uso eficiente de la memoria ni la conservación de la consistencia de los datos. Estas limitaciones se agravan ante operaciones de procesamiento intensivo, que pueden bloquear la interfaz y dificultar la interrupción segura de las tareas, comprometiendo la continuidad del trabajo experimental y la confiabilidad de los resultados.
Adicionalmente, la ausencia de mecanismos adecuados para gestionar los fallos y registrar los errores dificulta la identificación de sus causas y la recuperación de la aplicación. Dado que todos los componentes se ejecutan en un mismo proceso, un fallo puede comprometer el funcionamiento de toda la herramienta y ocasionar la pérdida del trabajo realizado durante la sesión. Estas limitaciones evidencian la necesidad de establecer condiciones que fortalezcan la confiabilidad de la aplicación y faciliten su mantenimiento.
A partir de este diagnóstico se definieron los requerimientos no funcionales de Gamma Lab 2.0, que traducen los hallazgos de la revisión técnica en condiciones verificables para su implementación. Los requerimientos se presentan en el orden de los módulos establecido en la sección de nomenclatura 3.1.

### 3.3.1. Categoría U: Usabilidad

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-U-01 | R02 | El sistema debe diferenciar los mensajes informativos, de advertencia y de error, usando el nivel de severidad que corresponda a cada situación. |

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-U-02 | R03 | El sistema debe mostrar el resultado de la operación en la misma pantalla en la que se realizo |
| NFR-U-03 | R04 | Toda operación que tome más de 1 segundo debe mostrar retroalimentación visual de progreso |
| NFR-U-04 | R06 | El sistema debe contar con iconos para cada funcionalidad importante |
| NFR-U-05 | R10 | EL sistema debe darle una bienvenida a usuarios nuevos |
| NFR-U-06 | R12 | El sistema debe comunicar el cierre de un proyecto |
| NFR-U-07 | R42 | El sistema debe mostrar una descripción emergente del propósito de cada control, en cada funcionalidad que se ejecute con un botón. |
| NFR-U-08 | R43 | El sistema debe mostrar en Home las funcionalidades de edición de la vista y de las mediciones de la señal. |
| NFR-U-09 | R44 | El sistema debe mostrar una pantalla de inicio mientras carga los recursos de arranque. |

Tabla 18: Requerimientos no funcionales de la categoría Usabilidad

### 3.3.2. Categoría P: Rendimiento

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-P-01 | R11 | El sistema debe liberar la memoria de la señal activa y sus derivados al cerrarse el proyecto activo, ya sea por cierre explícito o por la carga de una nueva señal. |
| NFR-P-02 | R45 | El sistema debe liberar, al salir de cada módulo, los elementos gráficos y los datos pesados que mantenía en memoria, sin limitarse a ocultarlos de la pantalla. |
| NFR-P-03 | R47 | El sistema debe cargar el archivo de señal en un tiempo máximo de 2 segundos, contados desde que inicia la lectura hasta que la señal aparece en pantalla, sobre el hardware de referencia del laboratorio. |
| NFR-P-04 | R48 | El sistema debe completar el cómputo de Wavelet sobre una señal en $\leq$ 10 segundos |
| NFR-P-05 | R49 | El sistema debe renderizar al menos 3 canales simultáneamente sin que la tasa de refresco caiga por debajo de 24 FPS durante interacción de zoom o desplazamiento, sobre el hardware de referencia del laboratorio. |

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-P-06 | R50 | El sistema debe completar el cambio entre módulos de análisis en $\leq$ 500 ms manteniendo la señal activa en memoria, sin recargarla desde disco. |
| NFR-P-07 | R51 | El sistema debe exportar tablas de resultados (CSV, Excel) de hasta 100,000 filas en menos de 2 segundos. |
| NFR-P-08 | R52 | El sistema debe liberar la memoria asociada a una señal previamente cargada en menos de 1 segundo tras confirmarse la carga de una nueva señal, evitando el crecimiento acumulativo |
| NFR-P-09 | R53 | El sistema debe completar su cierre, liberando los recursos gráficos, la memoria y los procesos que tenía abiertos, en menos de 3 segundos. |
| NFR-P-10 | R55 | El sistema debe mantener la interfaz responsiva, con respuesta $\leq$ 0.3 s ante las interacciones, incluso mientras se ejecuta en segundo plano un análisis de frecuencia o de tiempo-frecuencia. |
| NFR-P-11 | R56 | El sistema debe completar el promedio de todos los trials en $\leq$ 1 segundo para un archivo típico de hasta 50 trials. |
| NFR-P-12 | R57 | El sistema debe completar la exportación de una gráfica individual en un tiempo máximo de 1 segundo cuando el formato es una imagen rasterizada, y de 3 segundos cuando es vectorial. |
| NFR-P-13 | R59 | El sistema debe construir las representaciones gráficas de las señales y de los resultados mediante conversiones en bloque, sin recorrer los datos elemento por elemento. |
| NFR-P-14 | R71 | El sistema debe ejecutar a través del servicio de tareas del núcleo, fuera del hilo de la interfaz, toda operación de cómputo de los plugins: la carga de señal, el corte de trials, el filtrado, la eliminación de artefactos, los análisis de frecuencia individuales y promediados, el promedio de trials, el ERP y los análisis de tiempo-frecuencia individuales y promediados. |
| NFR-P-15 | R73 | El sistema debe limitar a 2000 puntos por serie y 200 curvas simultáneas en pantalla, submuestreando solo la representación visual sin alterar los datos. |
| NFR-P-16 | R74 | El sistema debe ejecutar el análisis PAC a través del servicio de tareas del núcleo y completar un comodulograma de hasta 20×20 pares de bandas en un tiempo máximo de 30 s, permitiendo su cancelación. |

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-P-17 | R81 | El sistema debe quedar listo para usarse en un tiempo máximo de 8 segundos desde que se ejecuta, sobre el hardware de referencia del laboratorio. |

Tabla 20: Requerimientos no funcionales de la categoría Rendimiento

### 3.3.3. Categoría R: Confiabilidad

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-R-01 | R46 | El sistema debe cancelar de forma cooperativa toda tarea del servicio de tareas cuando se detiene el plugin que la originó o cuando el usuario la cancela y, si la tarea no responde en un máximo de 2 segundos, debe desligarla, descartar su resultado y liberar la interfaz y la cola, de modo que la aplicación no se bloquee ni aplique un resultado obsoleto. |
| NFR-R-02 | R54 | El sistema debe sincronizar el acceso al estado compartido de la sesión cuando varias operaciones se ejecuten a la vez, de modo que un solo propietario escriba cada dato. |
| NFR-R-03 | R58 | El sistema debe retirar los detectores de eventos registrados sobre las gráficas al salir de cada módulo. |
| NFR-R-04 | R72 | El sistema debe reutilizar la representación gráfica ya construida de una señal mientras sus datos no cambien, y descartarla de forma explícita cuando se modifiquen. |
| NFR-R-05 | R79 | El sistema no debe modificar la señal cargada ni los trials originales. Toda operación de preprocesamiento debe registrar su resultado como un conjunto derivado, conservando disponible el conjunto de origen. |

Tabla 22: Requerimientos no funcionales de la categoría Confiabilidad

### 3.3.4. Categoría S: Soporte

| Código | N° Matriz | Especificación |
|---|---|---|
| NFR-S-01 | R39 | El sistema debe generar un ejecutable instalable que registre la extensión .glab en el sistema operativo, de modo que abrir un archivo de proyecto con doble clic inicie la aplicación. |
| NFR-S-02 | R67 | El sistema debe ofrecer una vía pública y única para leer, modificar e invalidar los trials y sus descartes, sin que ningún módulo alcance la estructura interna de los datos. |
| NFR-S-03 | R69 | El sistema debe registrar en el núcleo los servicios compartidos de exportación, medición y ajustes, y entregarlos a los módulos con una única instancia de cada uno, sin que ningún módulo los construya por su cuenta. |
| NFR-S-04 | R70 | El sistema debe ejecutar toda operación de cómputo pesada a través de un servicio de tareas registrado en el núcleo, que gestione la cola, la cancelación y la notificación de progreso. Se considera pesada toda operación que, con los datos típicos del laboratorio (una señal de hasta 1,8 millones de muestras por canal y hasta 60 trials de 30 501 muestras), pueda tardar más de 1 segundo en el hardware de referencia; R71 enumera además las operaciones que se despachan siempre por el servicio, sin importar su duración. |
| NFR-S-05 | R77 | El sistema debe permitir incorporar un nuevo plugin de análisis registrándolo en el núcleo y despachando su cómputo por el orquestador, sin modificar el núcleo ni los plugins existentes. |

Tabla 24: Requerimientos no funcionales de la categoría Soporte

# 4. PROCESO INGENIERÍA DE REQUERIMIENTOS

Esta sección describe el proceso, técnicas y métodos utilizados para la construcción y administración de los requerimientos del sistema GammaLab 2.0. El objetivo es garantizar que los requerimientos funcionales y no funcionales se obtengan, documenten, validen y mantengan de forma consistente durante todo el ciclo de vida del proyecto.

## 4.1 Enfoque General

La ingeniería de requerimientos en GammaLab 2.0 se concibe como un proceso iterativo y controlado, alineado con el modelo incremental del proyecto. Cada iteración permite refinar,

corregir o ampliar los requerimientos según la retroalimentación obtenida durante el análisis, diseño, implementación y pruebas.
El proceso involucra tanto al equipo de desarrollo como al cliente final, el Laboratorio de Señales, asegurando que las necesidades funcionales y los atributos de calidad se representen de manera verificable. Dado que GammaLab 2.0 es una versión evolutiva, el proceso parte del análisis de las limitaciones identificadas en la versión anterior y de los nuevos requerimientos planteados por el cliente.

## 4.2 Actividades del Proceso

El proceso seguido incluye las siguientes actividades principales:

- **Elicitación:** Identificación de necesidades mediante entrevistas con el cliente, análisis de las limitaciones de GammaLab 1.0 (SRS v1.1) y revisión técnica del dominio de señales EEG y análisis de acoplamiento cruzado de frecuencias.
- **Análisis y refinamiento:** Agrupación, priorización y normalización de requerimientos de acuerdo con los cuatro frentes de desarrollo de GammaLab 2.0: mejora de UX, paralelización del Kernel, nuevo módulo AC e integración de la paralelización con el módulo AC.
- **Documentación:** Registro en el presente SRS utilizando la estructura estándar IEEE 29148, garantizando claridad, unicidad y verificabilidad de cada requerimiento.
- **Validación:** Revisión con la directora del proyecto en su rol de Product Owner y con el cliente, mediante prototipos tempranos y sesiones de revisión semanales.
- **Trazabilidad:** Enlace de cada requerimiento con los módulos del sistema, el modelo de dominio y los criterios de medición definidos en las fichas de especificación.

## 4.3 Control de Cambios

Los requerimientos pueden modificarse a lo largo del desarrollo debido a hallazgos técnicos o retroalimentación del cliente. Para ello se implementa el siguiente proceso de control:

- Canal formal de solicitud de cambio registrado en el repositorio GitHub del proyecto.
- Evaluación técnica y de esfuerzo por parte del equipo de desarrollo.
- Aprobación por parte de la directora del proyecto (Product Owner).
- Actualización del SRS, de la distribución de requerimientos y de los artefactos afectados.

## 4.4 Artefactos Generados

| Artefacto | Descripción |
|---|---|
| SRS v1.0 | Presente documento de especificación de requerimientos. |
| Acta de requerimientos | Documento inicial que consolida los requerimientos levantados durante las sesiones de elicitación. |
| Distribución de requerimientos | Tabla de trazabilidad entre módulos y requerimientos funcionales. |
| PMP | Plan de Administración del Proyecto con cronograma, roles y gestión de riesgos. |
| Historial de versiones | Control de actualizaciones del SRS registrado en el encabezado del documento. |

Tabla 25: Artefactos generados durante el proceso de ingeniería de requerimientos

# 5. PROCESO VERIFICACIÓN

Esta sección describe el proceso aplicado para verificar y validar los requerimientos definidos en esta especificación. La verificación en GammaLab 2.0 se desarrolla en estrecha colaboración con la directora del proyecto en su rol de Product Owner y con el cliente, el Laboratorio de Señales de la Universidad Nacional de Colombia.

## 5.1 Participación del Cliente

El proceso de verificación cuenta con la supervisión de la directora del proyecto, quien actúa como enlace técnico con el Laboratorio de Señales. Su rol es fundamental para:

- Validar los requerimientos durante su construcción, asegurando que reflejen las necesidades reales del laboratorio.
- Proporcionar retroalimentación técnica desde la perspectiva del dominio EEG y del análisis de acoplamiento cruzado de frecuencias.
- Aceptar o rechazar funcionalidades implementadas en cada iteración de desarrollo.
- Asegurar que los resultados y herramientas generadas sean coherentes con los procedimientos experimentales del laboratorio.

## 5.2 Dinámica de Verificación

El proceso de verificación se desarrolla mediante ciclos iterativos de revisión. En cada ciclo se presentan los avances implementados, incluyendo:

- Funcionalidades desarrolladas en la iteración correspondiente.
- Correcciones a requerimientos previamente observados.
- Comportamientos de la interfaz y flujos completos de uso.
- Validación directa sobre datos reales del laboratorio, cuando aplica.

Durante estas sesiones, el cliente aprueba, solicita ajustes o rechaza funcionalidades según su alineación con los objetivos científicos del proyecto. Esta dinámica permite mantener los requerimientos actualizados, claros y técnicamente viables a lo largo del ciclo de desarrollo.

## 5.3 Mecanismo de Control y Seguimiento

Para garantizar trazabilidad completa, el equipo utiliza un flujo controlado mediante ramas en el repositorio oficial de GitHub. Cada avance del sistema se gestiona bajo las siguientes prácticas:

- Creación de ramas específicas por funcionalidad o corrección (`feature/nombre-funcionalidad`).
- Revisión interna por parte del equipo antes de su presentación al cliente.
- Retroalimentación del cliente y ajustes sobre la misma rama.
- Aprobación final y posterior integración (*merge*) al repositorio principal.

Este mecanismo permite asociar cada requerimiento con su evidencia de implementación y con el historial de revisiones, asegurando un proceso verificable y consistente.

## 5.4 Criterios de Verificación por Tipo de Requerimiento

Los criterios de verificación se organizan según la misma clasificación con la que se especificaron los requisitos en el capítulo *Requerimientos Específicos*: las cuatro áreas funcionales (**IO**, **UI**, **WV** y **PAC**) y las cuatro categorías no funcionales del modelo FURPS+ (**U**, **P**, **R** y **S**). De este modo, cada requisito enunciado en el documento tiene un criterio de verificación asociado y ninguno queda fuera del proceso.

| Tipo de requerimiento | Criterio de verificación |
|---|---|
| **Requerimientos Funcionales** (FR-XX-YY, 45 requisitos) | |
| **Módulo IO: Entrada/Salida** | Carga de cada formato soportado y reapertura de proyectos guardados, comprobando que la señal restaurada coincide con la original y que la sesión recupera los *trials*, los descartes y los análisis previos. Se ejecuta sobre archivos reales del laboratorio, con aprobación explícita del cliente. |
| **Módulo UI: Interacción general** | Recorrido guiado de las operaciones de edición de la señal y de gestión de *trials* sobre datos reales, verificando con el cliente que cada acción produce el resultado esperado y que es reversible. |
| **Módulo WV: Wavelet/Escalograma** | Comparación del escalograma contra resultados de referencia calculados con los mismos parámetros, evaluada por correlación, más revisión visual del cliente sobre señales de comportamiento conocido. |
| **Módulo PAC: Acoplamiento fase-amplitud** | Validación del comodulograma contra resultados de referencia conocidos y revisión de la gráfica por parte del cliente, cubriendo el acoplamiento intra-canal, el inter-canal y la coherencia de fase. |
| **Requerimientos No Funcionales** (NFR-X-YY, modelo FURPS+, 36 requisitos) | |
| **Categoría U: Usabilidad** | Pruebas de usabilidad con investigadores del laboratorio, en sesiones guiadas y no guiadas, observando si completan las tareas sin asistencia y recogiendo su valoración de los mensajes, los iconos y la retroalimentación de progreso. |
| **Categoría P: Rendimiento** | Medición instrumentada de tiempos de ejecución y de consumo de memoria frente a los umbrales que fija cada requisito, sobre el hardware de referencia y con señales del laboratorio. Los resultados se contrastan contra la línea base de GammaLab 1.0. |
| **Categoría R: Confiabilidad** | Pruebas automatizadas de los caminos críticos (cancelación de un análisis, cierre del proyecto a mitad de un cómputo y cambio de módulo con tareas en curso), verificando que no queden hilos trabajadores activos, detectores de eventos registrados ni datos derivados inconsistentes. |
| **Categoría S: Soporte** | Revisión de la arquitectura y prueba de extensión: incorporar un módulo de análisis nuevo registrándolo en el núcleo y comprobar que no exige modificar el núcleo ni los demás módulos. Se verifica además la instalación del ejecutable y el registro de la extensión `.glab`. |

Tabla 26: Criterios de verificación por tipo de requerimiento

## 5.5 Resultado del Proceso

La verificación continua con el cliente, junto con el control basado en ramas y revisiones iterativas, garantiza que:

- Cada requerimiento cuente con una validación explícita antes de considerarse implementado.
- Los cambios se gestionen de forma transparente y ordenada mediante el historial de Git.
- El sistema final cumpla con las expectativas científicas y funcionales del Laboratorio de Señales
- La trazabilidad entre requerimientos, diseño y código se mantenga durante todo el desarrollo del proyecto.

# 6. ANEXOS